

Proposed Resolution for NB Comment GB13-309 `atomic_ref<T>` is not convertible to `atomic_ref<const T>`

Document #: P3860R1 [[Latest](#)] [[Status](#)]
Date: 2025-11-03
Project: Programming Language C++
Audience: LEWG, LWG
Reply-to: Hui Xie
<hui.xie1990@gmail.com>
Damien Lebrun-Grandié
<lebrungrandt@ornl.gov>

Contents

- [1 Revision History](#)
 - [1.1 R1](#)
 - [1.2 R0](#)
- [2 Abstract](#)
- [3 Introduction](#)
- [4 Wording](#)
- [5 Implementation Experience](#)
- [6 Feature Test Macro](#)
- [7 References](#)

§ 1 Revision History

§ 1.1 R1

- Added `noexcept` specifier.
- Changed constraints wording to say “T and U are similar types” per LWG feedback
- Added converting constructor to specializations for integral types
- Added converting constructor to specializations for floating-point types
- Added converting constructor to specializations for pointer types

§ 1.2 R0

— Initial revision

§ 2 Abstract

This paper proposes a converting constructor for `atomic_ref` as the resolution for UK NB Comment GB13-309.

§ 3 Introduction

The UK NB Comment GB13-309 pointed out that `atomic_ref<T>` is not convertible to `atomic_ref<const T>`. [P3323R1] added cv qualifiers to `atomic` and `atomic_ref`. However, the conversion constructor of `atomic_ref` between different cv qualifiers is overlooked.

§ 4 Wording

Change [atomics.ref.generic.general] as follows:

```
constexpr explicit atomic_ref(T&);  
constexpr atomic_ref(const atomic_ref&) noexcept;  
template <class U>  
constexpr atomic_ref(const atomic_ref<U>&) noexcept;
```

Add a new entry to [atomics.ref.ops] after paragraph 8:

```
template <class U>  
constexpr atomic_ref(const atomic_ref<U>& ref) noexcept;
```

9 *Constraints:*

(9.1) — T and U are similar types ([conv.qual]), and

(9.2) — `is_convertible_v<U*, T*>` is true

10 *Postconditions:* `*this` references the object referenced by `ref`.

Change [atomics.ref.int] as follows:

```
constexpr explicit atomic_ref(integral-type&);  
constexpr atomic_ref(const atomic_ref&) noexcept;  
template <class U>  
constexpr atomic_ref(const atomic_ref<U>&) noexcept;
```

Change [atomics.ref.float] as follows:

```
constexpr explicit atomic_ref(floating-point-type&);  
constexpr atomic_ref(const atomic_ref&) noexcept;
```

```
template <class U>  
constexpr atomic_ref(const atomic_ref<U>&) noexcept;
```

Change `[atomics.ref.pointer]` as follows:

```
constexpr explicit atomic_ref(pointer-type&);  
constexpr atomic_ref(const atomic_ref&) noexcept;  
template <class U>  
constexpr atomic_ref(const atomic_ref<U>&) noexcept;
```

§ 5 Implementation Experience

[P3323R1] is currently being implemented in libc++ and adding the proposed constructor is under way.

§ 6 Feature Test Macro

[P3323R1] does not seem to mention Feature Test Macro

§ 7 References

[P3323R1] Gonzalo Brito Gadeschi. 2024-11-18. cv-qualified types in atomic and atomic_ref.

<https://wg21.link/p3323r1>